

AI FitTrack Backend Application

Comprehensive Technical Documentation, System Architecture, & API Specifications

1. Executive Overview & Description

The FitTrack AI backend is a secure, scalable, and AI-powered RESTful application developed to simplify workout tracking and provide intelligent fitness recommendations. Built using Node.js, Express.js, MongoDB, and Mongoose, the backend exposes REST APIs that enable secure user authentication, workout management, workout searching, and AI-assisted fitness guidance.

The application follows the Model-View-Controller (MVC) architecture, making the project modular, maintainable, and scalable. User authentication is implemented using JSON Web Tokens (JWT), while passwords are securely encrypted using bcrypt.js before storage in MongoDB. Every protected API validates the JWT token through middleware before allowing access to sensitive resources.

One of the key features of the project is its integration with Google Gemini AI, which provides personalized workout recommendations based on a user's age, fitness goals, and workout experience. The AI module also generates fitness insights by analyzing workout statistics such as total workouts completed, average workout duration, and calories burned.

2. Scenario-Based Case Study

Background

Rahul is a fitness enthusiast who exercises regularly but records his workouts manually using notebooks and spreadsheets. Although he maintains consistency in his fitness routine, he finds it difficult to organize workout history, monitor progress, and determine whether his exercise habits align with his fitness goals. He also lacks professional guidance when deciding which workouts are best suited for his current fitness level.

Problem Statement

- Difficulty maintaining a centralized record of workouts.
- Manual tracking leads to inaccurate workout history.
- No secure user authentication for personal fitness records.
- Unable to search previous workouts efficiently.
- No personalized workout recommendations.
- No insights regarding overall workout performance.
- Time-consuming manual calculation of calories burned and workout duration.

Solution & Outcome

The FitTrack AI provides a centralized backend system where users can securely register, authenticate, and manage workout records. Users can add, update, delete, retrieve, and search workouts through RESTful APIs. By integrating Google Gemini AI, the application analyzes user information and workout statistics to generate intelligent workout recommendations and personalized fitness insights.

The backend follows industry-standard security practices including JWT authentication, password hashing, middleware-based authorization, centralized error handling, and MongoDB schema validation.

User Usage

A registered user can create an account using the registration endpoint and securely log in using email and password. After successful authentication, the backend generates a JWT token that authorizes access to protected APIs.

Using authenticated APIs, users can:

- Create new workout records.
- View all previously completed workouts.
- Retrieve a workout by its unique identifier.
- Update existing workout information.
- Delete unwanted workout records.
- Search workouts using workout name, category, or workout date.
- Generate AI-powered workout recommendations.
- Obtain AI-generated fitness insights based on workout statistics.

AI Usage

The AI module communicates with Google Gemini using a dedicated service layer.

Users provide information such as:

- Age
- Fitness Goal
- Experience Level

Gemini processes this information and returns:

- Personalized workout plan
- Weekly exercise suggestions
- Beginner/Intermediate/Advanced recommendations
- Motivational guidance
- Safety tips

Similarly, users can submit workout statistics including:

- Total Workouts
- Average Workout Duration
- Total Calories Burned

The AI module analyzes these values and produces personalized fitness insights.

Outcome

The AI FitTrack backend significantly improves fitness tracking by eliminating manual record keeping and providing intelligent recommendations based on user activity. Secure authentication ensures that only authorized users can access personal workout data, while AI assistance helps users improve their exercise routines with personalized suggestions.

The modular MVC architecture enables easy maintenance and future scalability. The application can later be extended with additional features such as nutrition tracking, BMI calculation, wearable device integration, workout reminders, and fitness analytics dashboards without major architectural changes.

3. System Requirements

Software Requirements

Operating System: Windows 10/11, macOS, or Linux (cross-platform).

Node.js (v16+): Runtime ecosystem running server logic and routing infrastructure.

npm (v8+): Node package manager to control operational dependencies.

Express.js: Lightweight routing web framework to construct backend RESTful entry points. MongoDB:

Document NoSQL engine storing users, posts, categories, comments, and analytics metrics.

Postman / ThunderClient: API verification toolkit to assert schema validation responses across administrative route shields.

Hardware Requirements

Processor: Intel Core i5 (8th Gen or above) / AMD Ryzen 5 or equivalent.

RAM: 8 GB minimum (16 GB recommended for concurrent instances of MongoDB and processing layers).

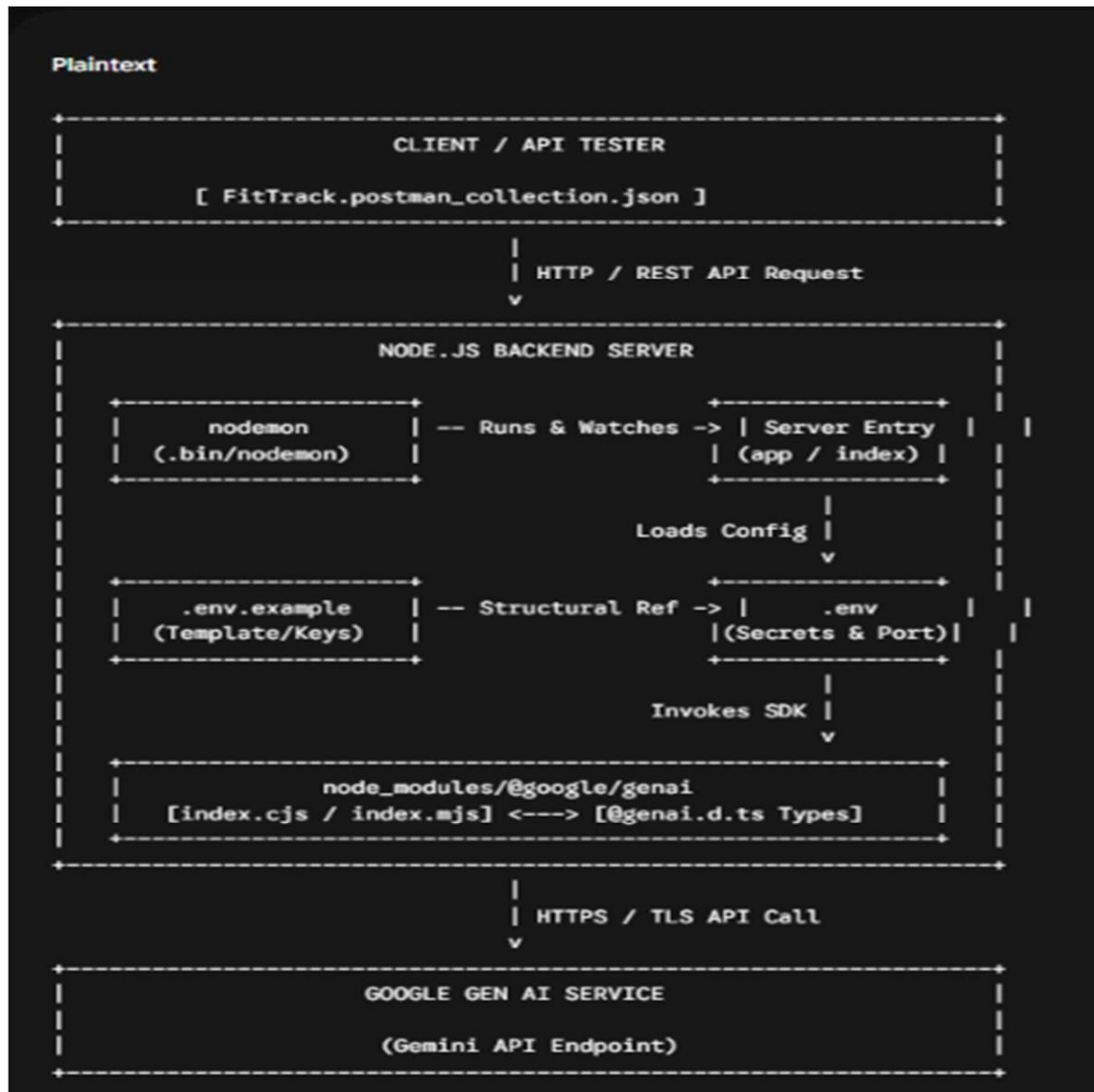
Storage: 1 GB of available disk workspace.

4. Project Architecture & Components

The FitTrack AI follows a layered architecture based on the MVC (Model–View–Controller) design pattern.

The application separates routing, business logic, database interaction, middleware, and AI services into independent modules, making the project easier to maintain and scale.

Project Architecture



Core

Component Definitions

- Client Layer: Entry point of application (React, Mobile App, Postman). Sends HTTP requests and receives JSON responses.

Responsibilities

- Sends API requests.
- Displays workout information.
- Stores JWT token after login.
- Invokes AI recommendation APIs.
- Displays success and error responses.

APIs Accessed

- Authentication APIs

- ☐ Workout APIs
- ☐ AI Recommendation APIs
- ☐ AI Fitness Insight APIs

2. Express Server Gateway: Central gateway listening for incoming requests, configuring middleware, parsing JSON bodies, and enabling CORS.

Responsibilities

- ☐ Starts HTTP server
- ☐ Parses JSON requests
- ☐ Enables Cross-Origin Resource Sharing (CORS)
- ☐ Registers application routes
- ☐ Connects middleware
- ☐ Handles centralized errors

Example Responsibilities

- ☐ Receive HTTP Request
- ☐ Validate Request Format
- ☐ Forward Request to Routes
- ☐ Return JSON Response

3. Authentication Middleware: Protects secured APIs by reading Authorization headers, verifying JWT tokens, and decoding user claims.

Responsibilities

- ☐ Read Authorization Header
- ☐ Extract JWT Token
- ☐ Verify JWT Signature
- ☐ Decode User Information
- ☐ Attach User Information to Request
- ☐ Reject Unauthorized Requests

Benefits

- ☐ Prevents unauthorized access
- ☐ Secures workout information
- ☐ Allows only authenticated users to access protected APIs
- ☐ Eliminates repeated authentication logic inside controllers

4. Route Layer: Defines API endpoints and maps incoming HTTP requests to corresponding controller methods.

5. Controller Layer: Contains core business logic, inputs validation, service invocation, and structured response generation.

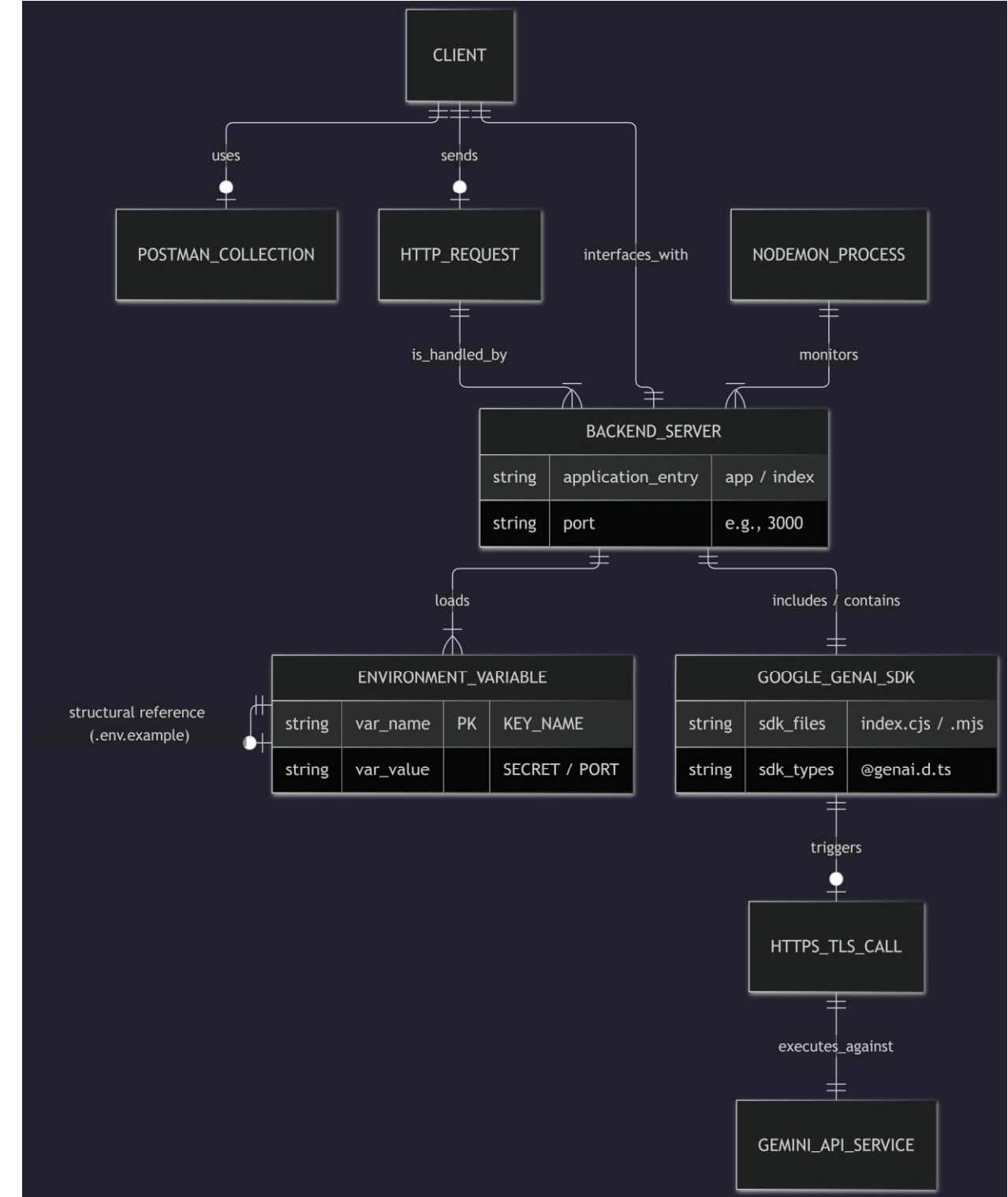
6. Service Layer: Encapsulates reusable services including Gemini AI integration, JWT token creation, and password hashing.

7. Database Layer: MongoDB NoSQL database managed via Mongoose schemas for Users and Workouts collections.

5. Database Schemas & Entity Relationship

The AI FitTrack backend consists of two primary collections that are logically connected through a one-to-many relationship (One User can create Many Workouts).

User Entity Schema (`models/User.js`)



Field	Type	Constraints / Description
_id	ObjectId	Primary Key

name	String	Required, Trimmed
email	String	Required, Unique, Lowercase, Regex Validated
password	String	Required, Encrypted using bcrypt.js (Min 6 chars)
createdAt	Date	Automatic Timestamp
updatedAt	Date	Automatic Timestamp

Workout Entity Schema (`models/Workout.js`)

Field	Type	Constraints / Description
_id	ObjectId	Primary Key
user	ObjectId	Foreign Key referencing User Collection
workoutName	String	Required, Trimmed
category	String	Required (Cardio, Strength Training, Yoga, Running, Cycling, Walking)
duration	Number	Required, Min: 1 minute
caloriesBurned	Number	Required, Min: 0
workoutDate	Date	Default: Date.now
createdAt	Date	Automatic Timestamp

6. API Endpoints & Specifications

HTTP Method	Endpoint	Protection	Description
POST	/api/auth/register	Public	Register new user account & return JWT
POST	/api/auth/login	Public	Authenticate credentials & return JWT
GET			/api/auth/profile Protected

			(JWT) Retrieve authenticated user details
POST	/api/workouts	Protected (JWT)	Create a new workout record
GET	/api/workouts	Protected (JWT)	Get all workouts for authenticated user
GET	/api/workouts/:id	Protected (JWT)	Get specific workout by ID
PUT	/api/workouts/:id	Protected (JWT)	Update existing workout by ID
DELETE	/api/workouts/:id	Protected (JWT)	Delete specific workout record
POST	/api/ai/workoutrecommendation	Protected (JWT)	Generate personalized workout plan via Gemini AI
POST	/api/ai/fitness-insights	Protected (JWT)	Generate performance insights via Gemini AI

Key Features

The FitTrack AI provides a complete backend solution for workout tracking and AI-assisted fitness guidance.

1. Secure Authentication

The application implements JWT-based authentication.

Features include:

- ☐ User Registration
- ☐ Secure Login
- ☐ Password Encryption
- ☐ JWT Generation
- ☐ Protected APIs

2. Workout Management

Users can perform complete CRUD operations.

Supported Operations

- ☐ Add Workout
- ☐ View Workout
- ☐ Update Workout
- ☐ Delete Workout

Each workout is linked to its authenticated user.

3. Workout Search

Users can search workouts using:

- ☐ Workout Name
- ☐ Workout Category
- ☐ Workout Date

This improves accessibility of workout history.

4. Google Gemini Integration

The backend communicates with Google Gemini AI to generate intelligent workout recommendations.

Input

- ☐ Age
- ☐ Fitness Goal
- ☐ Experience Level Output

- ☐ Weekly Workout Plan
- ☐ Suggested Exercises
- ☐ Training Tips
- ☐ Safety Recommendations

5. AI Fitness Insights

The AI module analyzes workout statistics.

Input

- ☐ Total Workouts

- ☐ Average Duration
- ☐ Calories Burned

Output

- ☐ Performance Analysis
- ☐ Improvement Suggestions
- ☐ Motivational Advice
- ☐ Fitness Progress Summary

6. Centralized Error Handling

The application returns meaningful JSON error responses for:

- ☐ Invalid Credentials
- ☐ Missing Authentication Token
- ☐ Validation Errors
- ☐ Database Errors
- ☐ Internal Server Errors

7. RESTful API Design

The backend follows REST principles.

Supported HTTP Methods

- ☐ GET
- ☐ POST
- ☐ PUT
- ☐ DELETE

Responses are returned in JSON format.

8. Modular Architecture

The project separates responsibilities into

- ☐ Models

- ☐ Controllers
- ☐ Routes
- ☐ Middleware
- ☐ Services
- ☐ Configuration

This improves maintainability and scalability.

Roles and Responsibilities

The AI FitTrack backend supports one primary role.

Authenticated User

Responsibilities

- ☐ Register an account.
- ☐ Login securely.
- ☐ View profile information.
- ☐ Add workout records.
- ☐ View workout history.
- ☐ Search workouts.
- ☐ Update workouts.
- ☐ Delete workouts.
- ☐ Request AI workout recommendations.
- ☐ Request AI fitness insights.

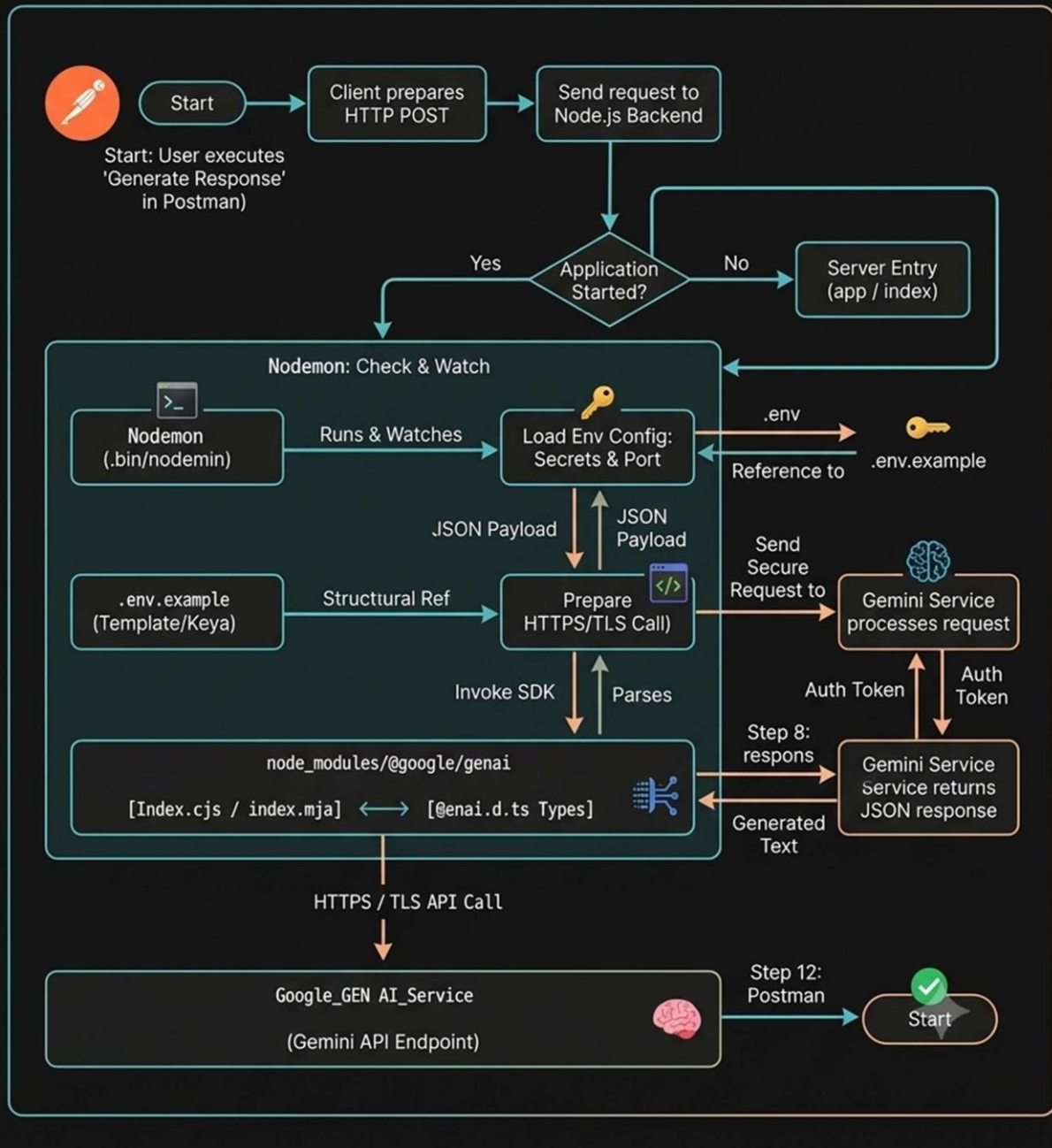
Permissions

Operation	Permission

Register	Yes
Login	Yes
View Profile	Yes
Add Workout	Yes
Get Workouts	Yes
Search Workouts	Yes
Update Workout	Yes
Delete Workout	Yes
AI Recommendation	Yes
Fitness Insights	Yes

4. Detailed User & Request Flow Diagram

This diagram traces the exact steps and interactions as a user (represented by the Postman collection) initiates an AI completion request and receives a response.

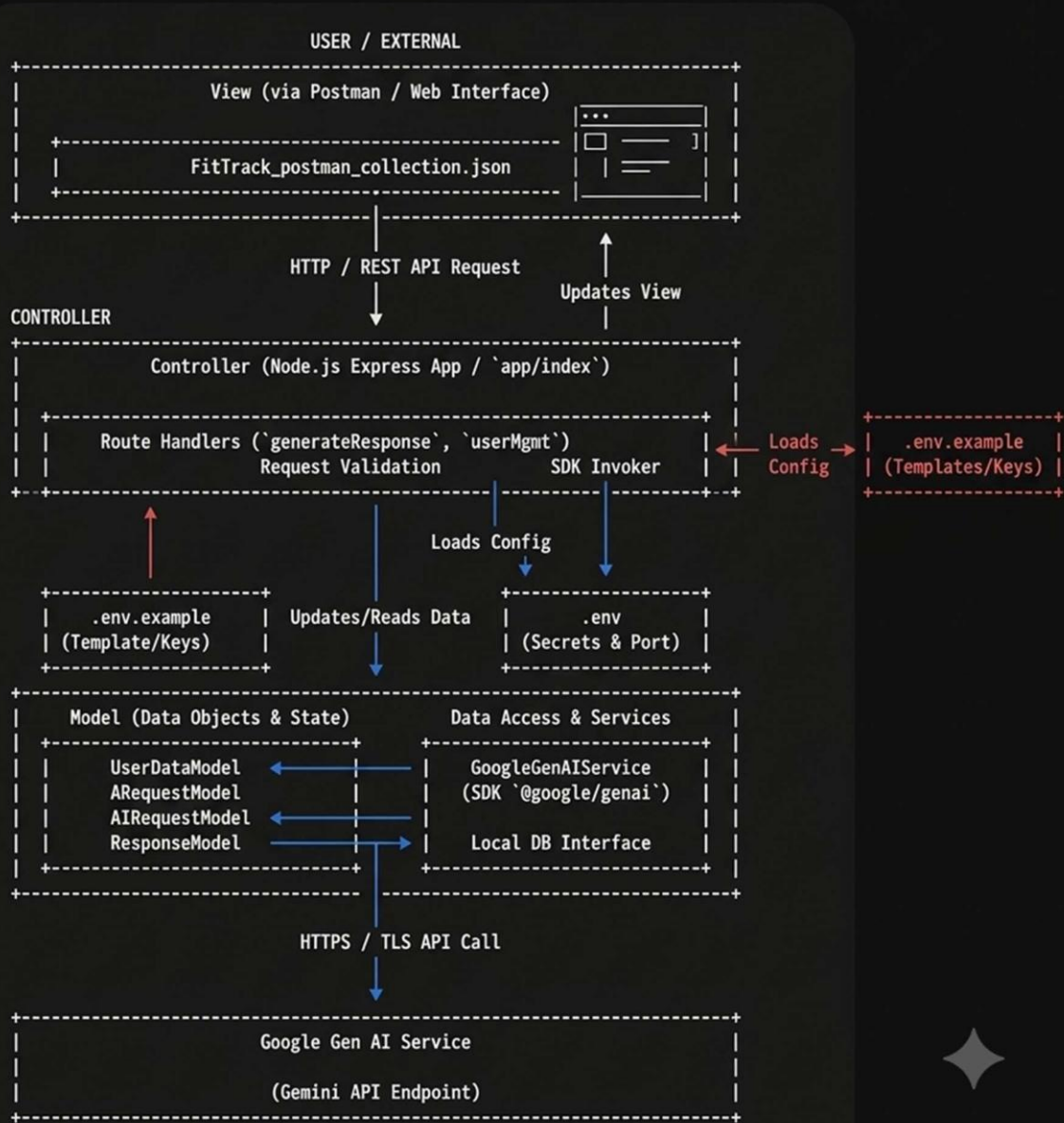


MVC Architecture Pattern

The FitTrack AI is designed using the Model–View–Controller (MVC) architecture pattern. This architecture separates application logic into independent components, improving code organization, maintainability, and scalability.

FitTrack MVC Architecture Pattern Diagram

This diagram shows the explicit separation of concerns between Model, View, and Controller for the FitTrack system, as they process client requests and interact with environment settings and the Google GenAI service.



Model Layer (Data Layer)

The Model Layer defines the structure of the application's data using Mongoose schemas. It communicates directly with MongoDB and performs database operations.

Models in the project include:

- User Model
- Workout Model

Responsibilities:

- Schema Definition
- Data Validation
- CRUD Operations
- Collection Mapping

View Layer (API Interface)

Since AI FitTrack is a backend REST API, there are no traditional UI templates. The View Layer is represented by API consumers such as React applications, mobile applications, or Postman, which interact with the backend through HTTP requests and receive JSON responses.

Responsibilities:

- Send Requests
- Receive JSON Responses
- Display Data
- Handle Client-side Interaction

Controller Layer (Business Logic)

The Controller Layer acts as the intermediary between the View and Model. It processes incoming requests, validates input, invokes services and models, and sends structured JSON responses back to the client.

Responsibilities:

- Request Validation
- Business Logic Execution

- Database Interaction
- AI Service Invocation
- Response Generation

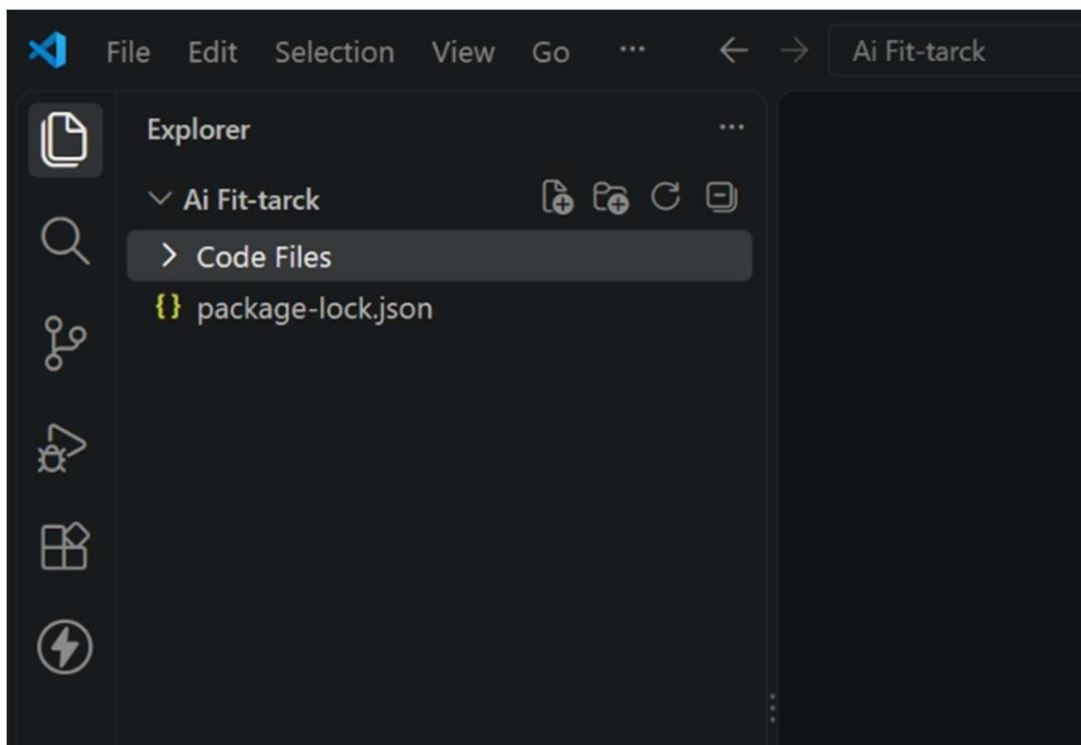
The separation of these three layers ensures loose coupling, simplifies testing, enhances code readability, and makes the application easier to extend with future features.

7. Step-by-Step Setup Guide

Step 1: Create the Project Folder

Create a new folder with the project name.

AI-FitTrack-API

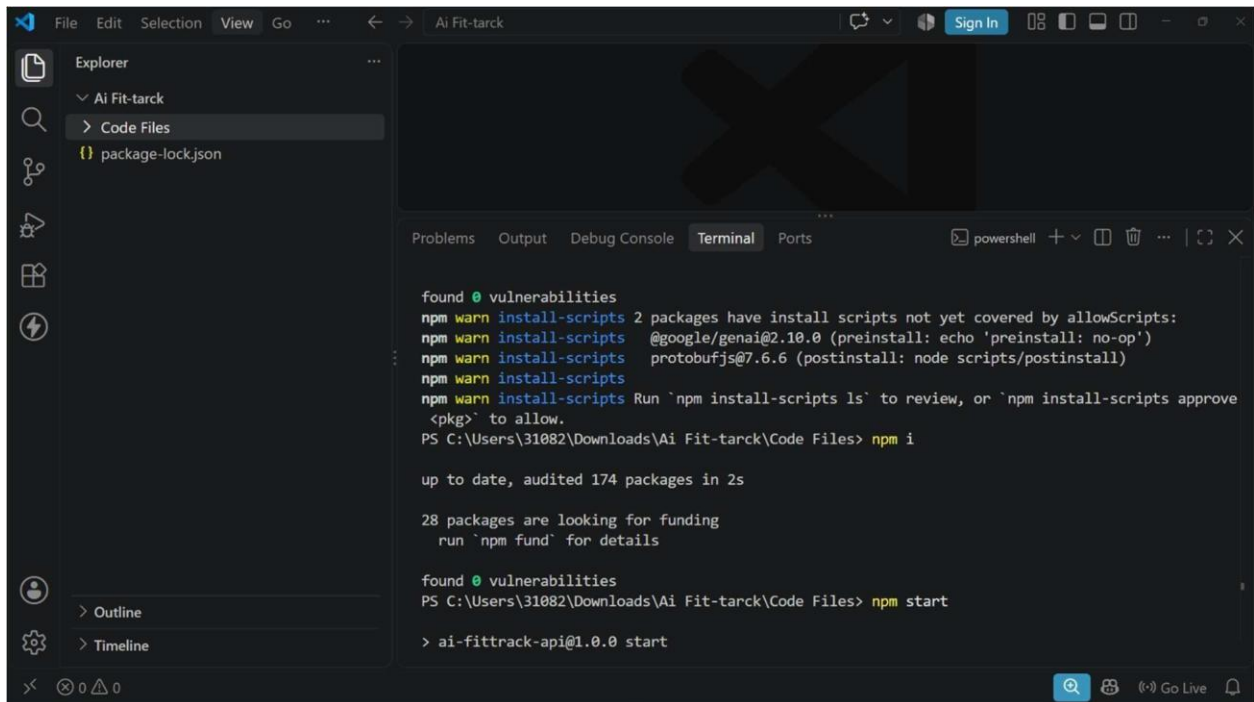


Step 2: Project Initialization

Create the project directory and initialize a Node.js project: `mkdir FitTrack-Backend`

`cd FitTrack-Backend`

`npm init -y`



Step 3: Install Dependencies

Install the official Google Gen AI SDK along with core web framework dependencies:
`npm install @google/genai express dotenv`

`npm install --save-dev nodemon`

Step 4: Environment Configuration

Create a `.env.example` file to document required variables:

`PORT=5000`

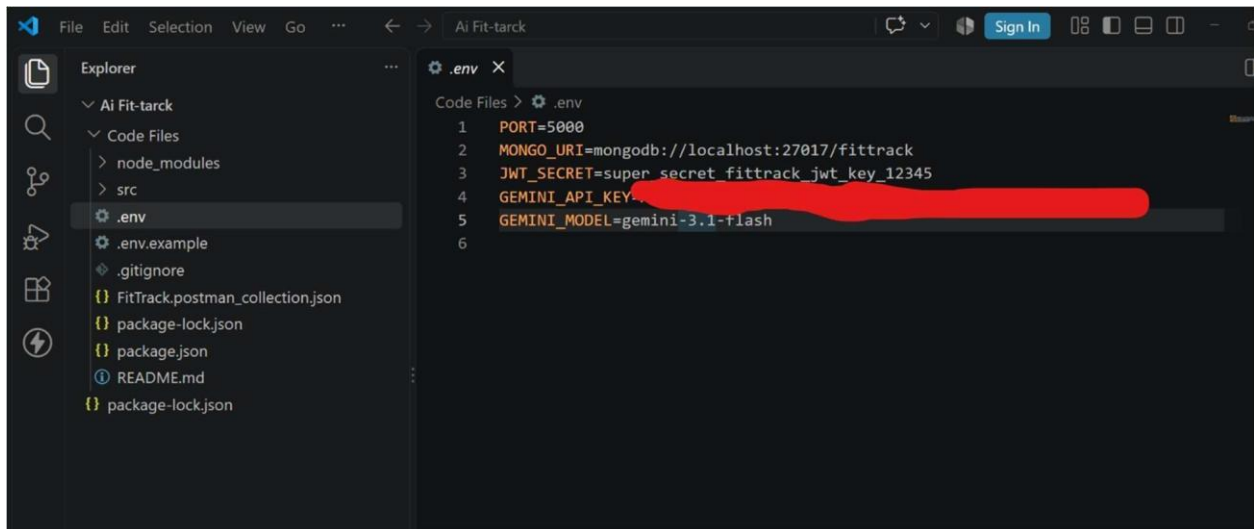
`GEMINI_API_KEY=your_gemini_api_key_here`

Duplicate `.env.example` to create `.env` and insert your API key:

```text`

`PORT=5000`

`GEMINI_API_KEY=AIzaSy..`



### Step 5: Package Scripts

Update package.json to configure start scripts and runtime behavior: {

```
"name": "fittrack-backend",

"version": "1.0.0",

"main": "src/app.js",

"type": "module",

"scripts": {

 "start": "node src/app.js",

 "dev": "nodemon src/app.js"

}
```

## Code Implementation

Service Layer (src/services/geminiService.js)

This layer directly interfaces with the @google/genai SDK using the GoogleGenAI class. import { GoogleGenAI } from '@google/genai';

```
class GeminiService {
```

```
 constructor() {
```

```

 this.ai = new GoogleGenAI({
 apiKey: process.env.GEMINI_API_KEY,

 });

 }

 /**
 * Generates a fitness recommendation based on user prompt.
 * @param {string} prompt - User health data and query.
 * @returns {Promise<string>} Generated text response.
 */

 async generateFitnessAdvice(prompt) {
 try
 {

 const response = await this.ai.models.generateContent({

 model: 'gemini-2.5-flash',

 contents: prompt,

 config: {

 systemInstruction: 'You are an expert fitness coach and nutritionist for the FitTrack platform.
Provide clear, concise, and safe fitness advice.',

 temperature: 0.7,

 },

 });

 return response.text;

 } catch (error) {

 console.error('Gemini Service Error:', error);

 throw new Error('Failed to generate fitness insights.');
```

```
}
export default new GeminiService();
```

### Controller Layer (src/controllers/aiController.js)

Handles incoming HTTP requests, performs basic validation, and routes requests to the Gemini service.  
import geminiService from '../services/geminiService.js';

```
export const getFitnessAdvice = async (req, res) => {

 try {

 const { prompt } = req.body;

 if (!prompt || typeof prompt !== 'string') {
|---|

 success: false,

 error: 'A valid "prompt" string is required in the request body.',

 });

 }

 const advice = await geminiService.generateFitnessAdvice(prompt);

 return res.status(200).json({

 success: true,

 data: {

 recommendation: advice,

 },
 });
} catch (error) {

 return res.status(500).json({

 success: false,

 error: error.message || 'Internal Server Error',

 });
}
```

```
});

}
};
```

### Express Routes (src/routes/aiRoutes.js)

Defines the endpoint routes for AI operations.

```
import { Router
} from 'express'; import { getFitnessAdvice } from
'../controllers/aiController.js';
```

```
const router = Router();
```

```
router.post('/generate-advice', getFitnessAdvice);
```

```
export default router;
```

### ### Server Entry Point (src/app.js)

Initializes express middleware, hooks up routes, and starts the listener.

```
````javascript
```

```
import express from 'express';
```

```
import dotenv from 'dotenv';
```

```
import aiRoutes from './routes/aiRoutes.js';
```

```
dotenv.config();
```

```
const app = express();
```

```
const PORT = process.env.PORT || 5000;
```

```
// Middleware
```

```
app.use(express.json());
```

```
// Routes
```

```
app.use('/api/v1/ai', aiRoutes);
```

```
// Health Check Endpoint
```

```
app.get('/health', (req, res) => { res.status(200).json({ status:  
'OK', service: 'FitTrack Backend' });
```

```
});  
  
app.listen(PORT, () => {  
  
  console.log(`FitTrack server running on http://localhost:${PORT}`); });
```

Installing Mongoose

Install Mongoose using npm.

```
npm install mongoose
```

Mongoose acts as the bridge between the Node.js application and MongoDB by providing schema definitions, validation, and database methods.

MongoDB Connection

The database connection is configured inside the config/db.js file. This module establishes a connection to the MongoDB server when the application starts

```
const mongoose = require('mongoose');  
  
const connectDB = async () => {  
  try {  
    const conn = await mongoose.connect(process.env.MONGO_URI);  
    console.log(`MongoDB Connected: ${conn.connection.host}`);  
  } catch (error) {  
    console.error(`Error connecting to MongoDB: ${error.message}`);  
    process.exit(1);  
  }  
};  
  
module.exports = connectDB;
```

Mongoose Schemas

The AI FitTrack backend contains two primary Mongoose schemas:

1. User Schema

2. Workout Schema

Schemas define the structure, validation rules, and relationships for MongoDB documents.

1. User Schema (models/User.js)

The User schema stores authentication information for registered users.

```
src > models > JS User.js > ...
1  const mongoose = require('mongoose');
2  const bcrypt = require('bcryptjs');
3
4  const userSchema = new mongoose.Schema(
5  {
6    name: {
7      type: String,
8      required: [true, 'Please add a name'],
9      trim: true,
10   },
11   email: {
12     type: String,
13     required: [true, 'Please add an email'],
14     unique: true,
15     trim: true,
16     lowercase: true,
17     match: [
18       /^\.w+([\.-]?\w+)*@\w+([\.-]?\w+)*(\.w{2,3})+$/,
19       'Please add a valid email address',
20     ],
21   },
22   password: {
23     type: String,
24     required: [true, 'Please add a password'],
25     minlength: 6, 'password must be at least 6 characters'],
26   },
27   },
28   {
29     timestamps: true,
30   }
31 );
32
33 // Pre-save password hashing hook
34 userSchema.pre('save', async function (next) {
35   if (!this.isModified('password')) {
36     return next();
37   }
38   const salt = await bcrypt.genSalt(10);
39   this.password = await bcrypt.hash(this.password, salt);
40   next();
41 });
42
43 // Compare password method
44 userSchema.methods.matchPassword = async function (enteredPassword) {
45   return await bcrypt.compare(enteredPassword, this.password);
46 };
47
48 module.exports = mongoose.model('User', userSchema);
```

2. Workout Schema (models/Workout.js)

The Workout schema stores workout information created by authenticated users.

```

src > models > JS Workout.js > workoutSchema > category > enum > values
1  const mongoose = require('mongoose');
2
3  const workoutSchema = new mongoose.Schema(
4  {
5    workoutName: {
6      type: String,
7      required: [true, 'Please add a workout name'],
8      trim: true,
9    },
10   category: {
11     type: String,
12     required: [true, 'Please select a workout category'],
13     enum: {
14       values: [
15         'Cardio',
16         'Strength Training',
17         'Yoga',
18         'Running',
19         'Cycling',
20         'Walking',
21       ],
22       message: '{VALUE} is not a supported workout category',
23     },
24   },
25   duration: {
26     type: Number,
27     required: [true, 'Please add the workout duration in minutes'],
28     min: [1, 'Duration must be at least 1 minute'],
29   },
30   caloriesBurned: {
31     type: Number,
32     required: [true, 'Please specify calories burned'],
33     min: [0, 'Calories burned cannot be negative'],
34   },
35   workoutDate: {
36     type: Date,
37     default: Date.now,
38   },
39   user: {
40     type: mongoose.Schema.Types.ObjectId,
41     ref: 'User',
42     required: true,
43   },
44 },
45 {
46   timestamps: true,
47 }
48 );
49
50 module.exports = mongoose.model('Workout', workoutSchema);

```

8. Project Execution

- Install Postman or ThunderClient (VS Code extension).
- Start the backend server:

“npm start”

API Testing

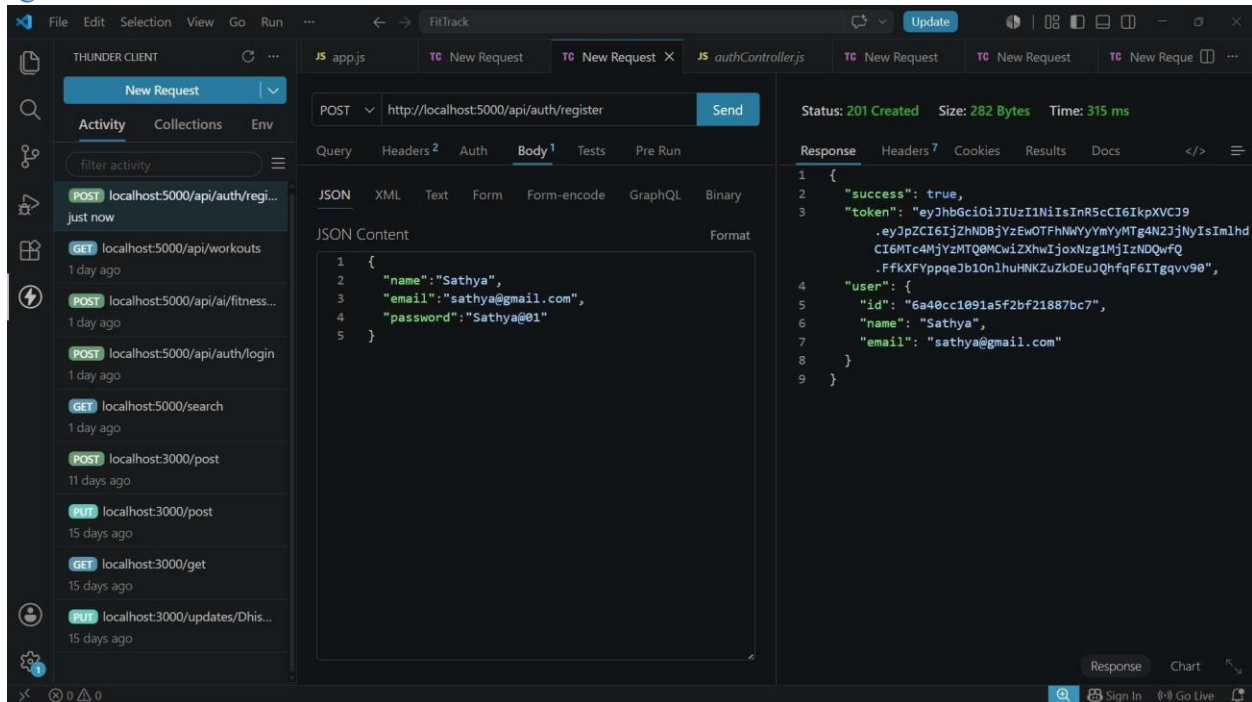
API testing ensures that every endpoint functions correctly and returns the expected response. The FitTrack AI is tested using Postman, allowing developers to verify request handling, authentication, database operations, and AI integration.

Testing is performed in the following order:

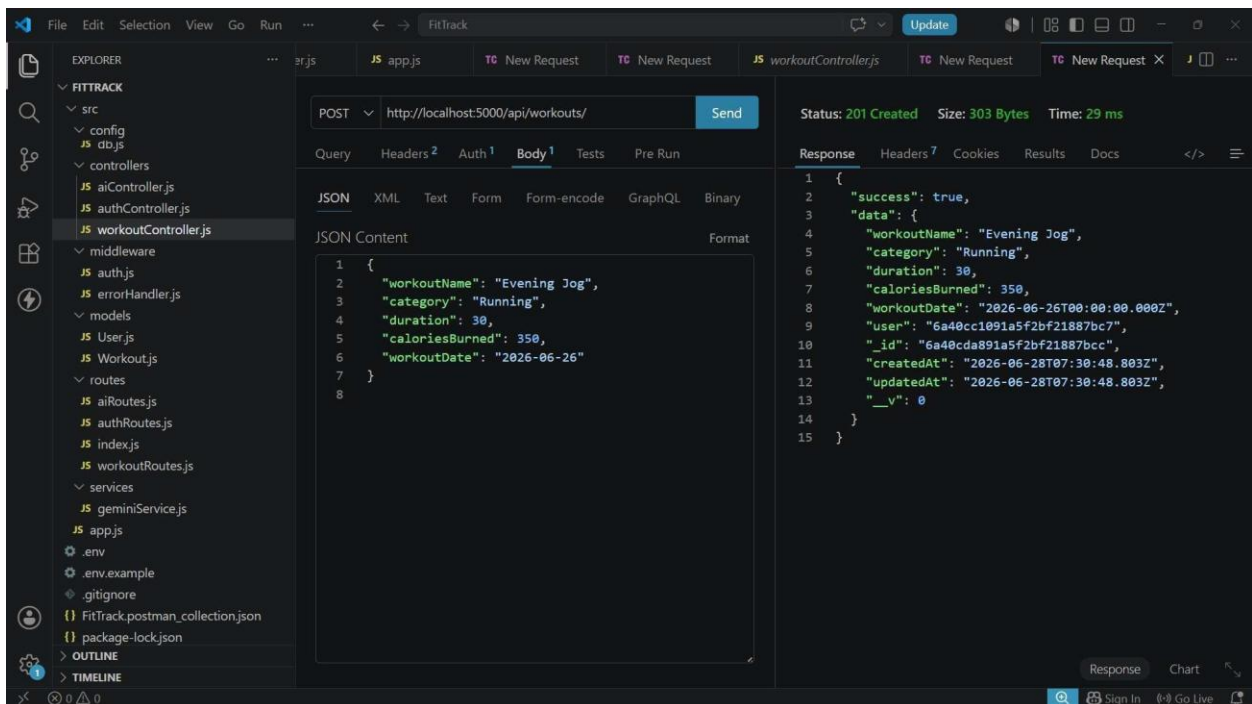
1. Authentication APIs
2. Workout APIs
3. AI APIs

Authentication API Testing

Register User

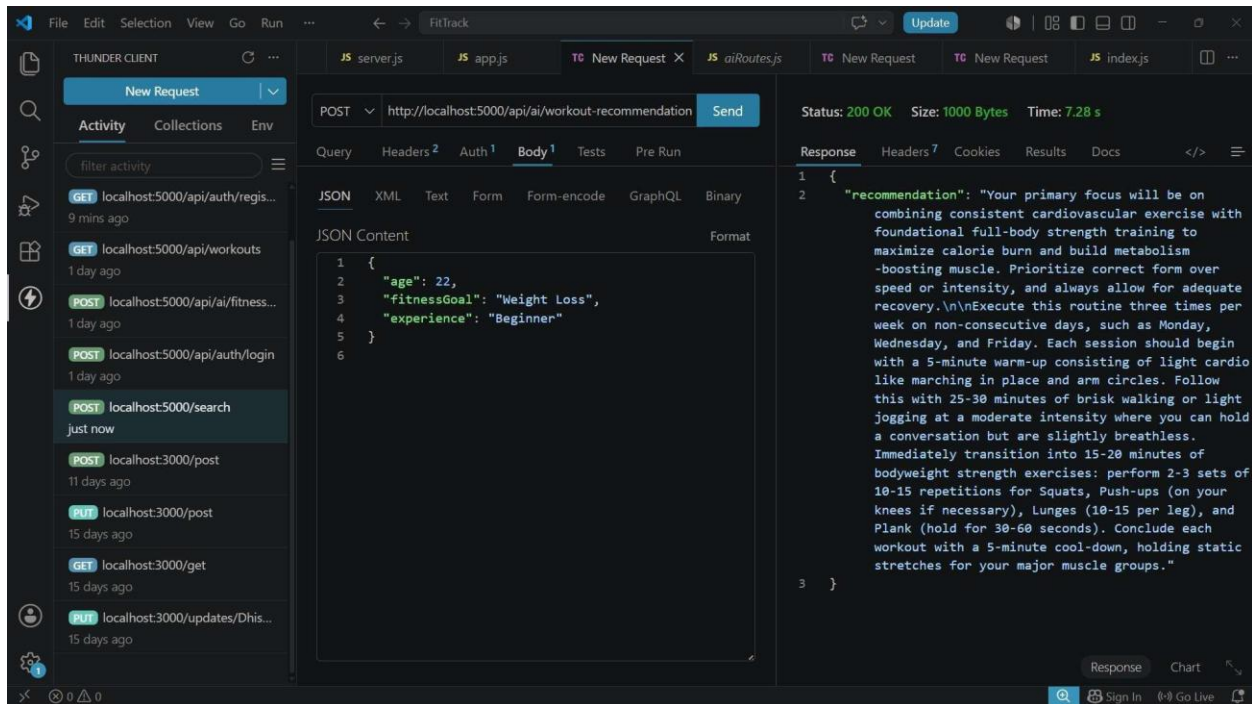


Login User



AI API Testing

Workout Recommendation



9. External Project Resources & Links

Backend Demo Video:

https://drive.google.com/drive/folders/17SgytRG_FD95BUglAwoBthNBUYfp0KhR?usp=sharing

API Testing Video:

<https://drive.google.com/file/d/1Xg6vdBc2zUD2b2c70RJYRfccPIWzU1nr/view?usp=sharing>

Github Project link:

<https://github.com/6469-ctrl/TN-NM-AI-AUGMENTED-BACKEND-PROJECT>